

DBSE AND DBD

Aim: Spring Boot with Rest API and CRUD Operations

Prerequisites:

General Idea on Spring Boot MVC and Form Handling
General Idea on Spring Data JPA

Description:

Develop a Spring Boot web application to manage a list of products in a warehouse. The application should handle CRUD operations to manage product details such as Product ID, Name, Description, Price, and Quantity. The application should include features to add new products, display a list of all products, update existing product details, and delete products from the database. Use Spring Web MVC for handling HTTP requests, Spring Data JPA for database interactions. Ensure the application is configured to connect to a MySQL/PostgreSQL database and implement both setter-based or constructor-based dependency injections to manage service and repository layers effectively.

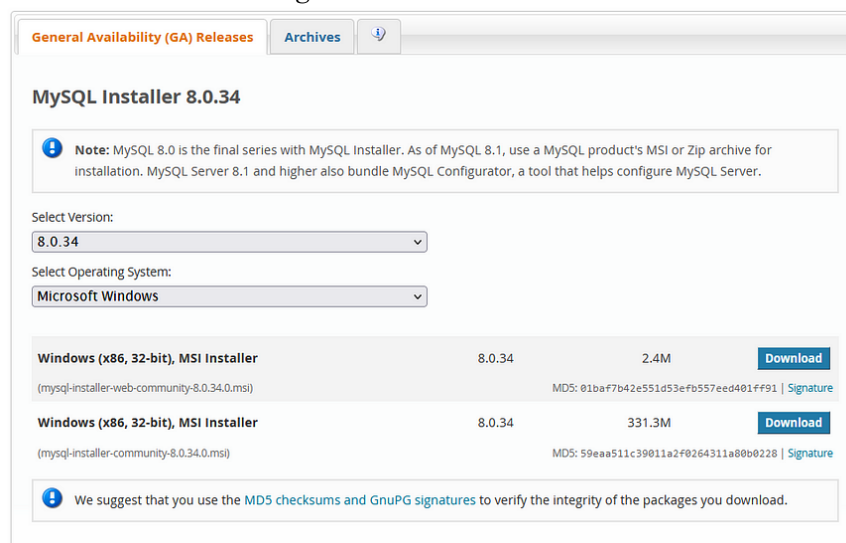
Download & install Mysql and MySQL Workbench:

MySQL is one of the most popular open-source relational database management systems, and MySQL Workbench is a powerful graphical tool for managing and interacting with MySQL databases. If you're looking to set up MySQL and MySQL Workbench on your Windows machine, this step-by-step guide will walk you through the installation process.

Step 1: Download MySQL Installer

- Open your web browser and navigate to the MySQL website at <https://dev.mysql.com/downloads/installer/>.
- Scroll down to the "MySQL Installer" section, make sure "Microsoft Windows" is selected under Select Operating System and click on the "Download" button for the MySQL Installer MSI (both the 2.4M and 331.3M version will work)

Note: This guide uses screenshots that might be hard to read on mobile.



On the next page, you'll be prompted to log in or create a MySQL account. You can choose "No thanks, just start my download."

Login Now or Sign Up for a free account.

An Oracle Web Account provides you with the following advantages:

- Fast access to MySQL software downloads
- Download technical White Papers and Presentations
- Post messages in the MySQL Discussion Forums
- Report and track bugs in the MySQL bug system

Login »
using my Oracle Web account

Sign Up »
for an Oracle Web account

MySQL.com is using Oracle SSO for authentication. If you already have an Oracle Web account, click the Login link. Otherwise, you can sign up for a free account by clicking the Sign Up link and following the instructions.

[No thanks, just start my download.](#)

Step 2: Install and Configure MySQL

- Locate the installer file you just downloaded (e.g., mysql-installer-web-community-x.x.xx.x.msi) and double-click it to run the installer.



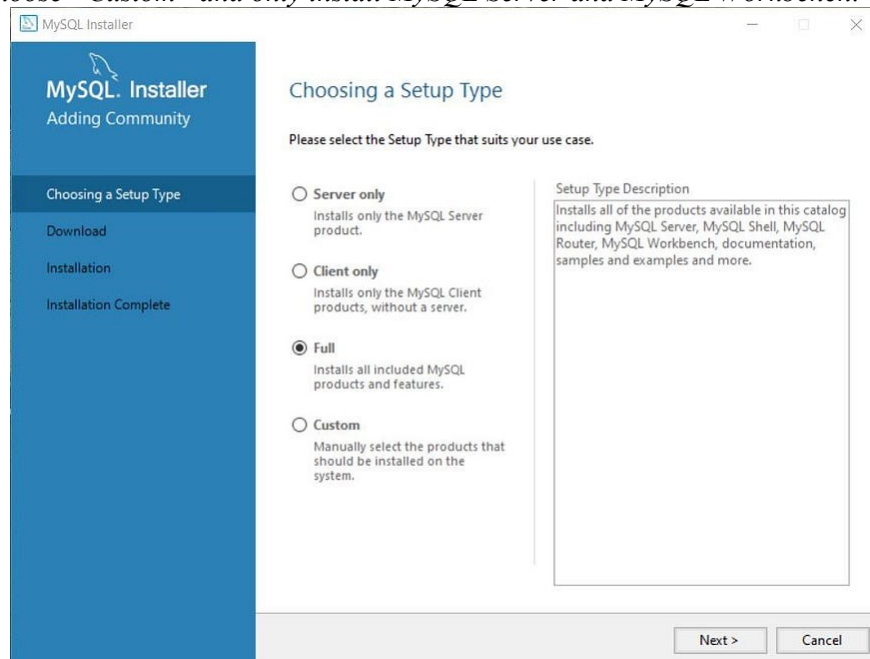
mysql-installer-web-community-8.0.34.0.msi

2,4 MB

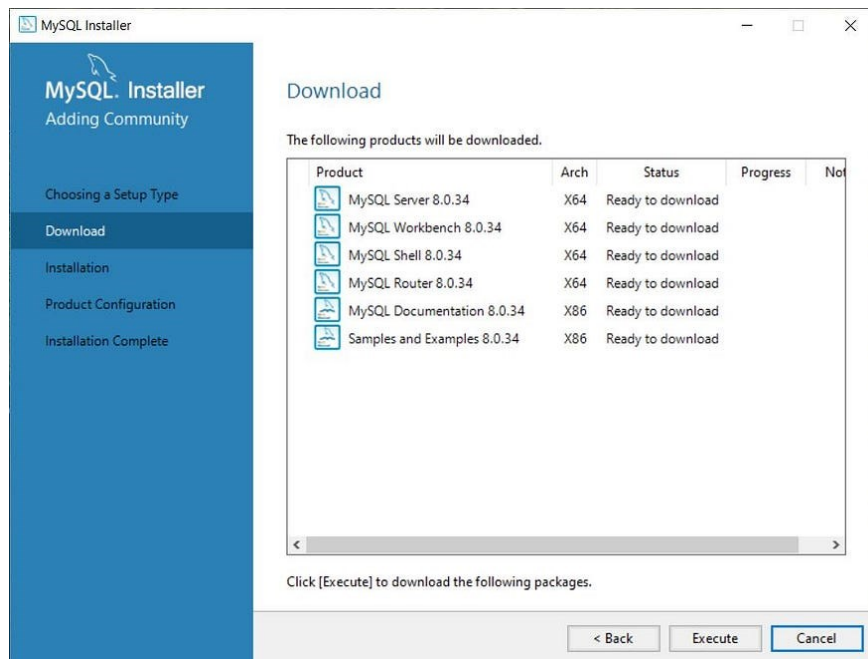


- In the installer window, choose “Full” under “Setup Type.” This option will install MySQL Server, MySQL Workbench and other components, such as documentation and examples. Then click “Next.”

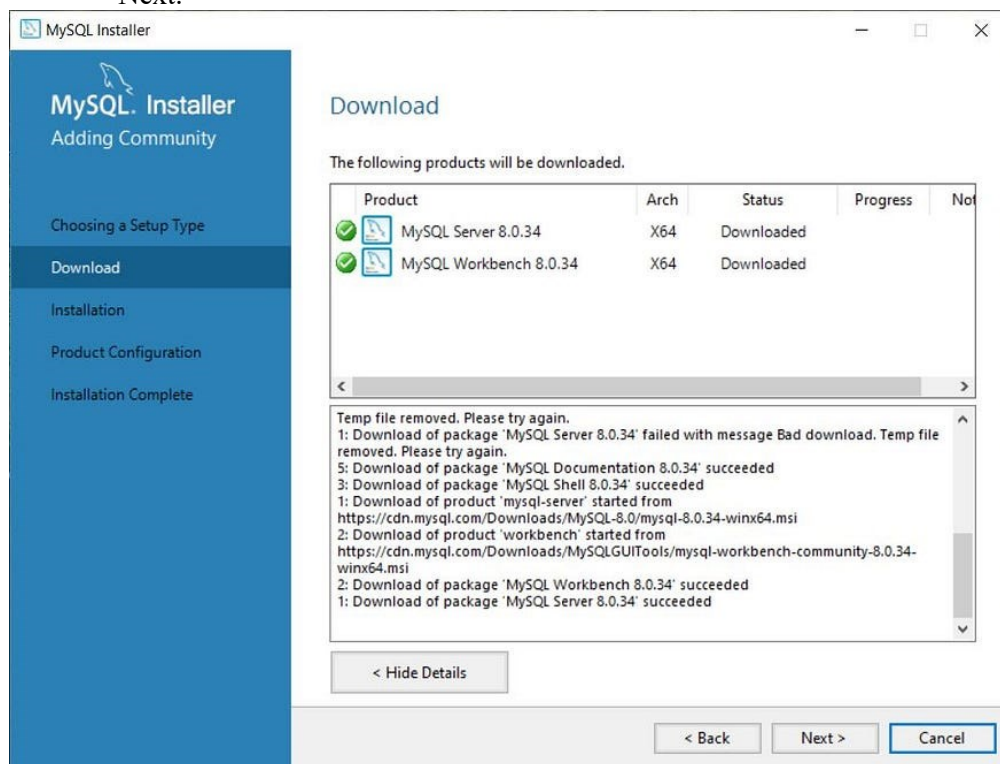
You can also choose “Custom” and only install MySQL Server and MySQL Workbench.



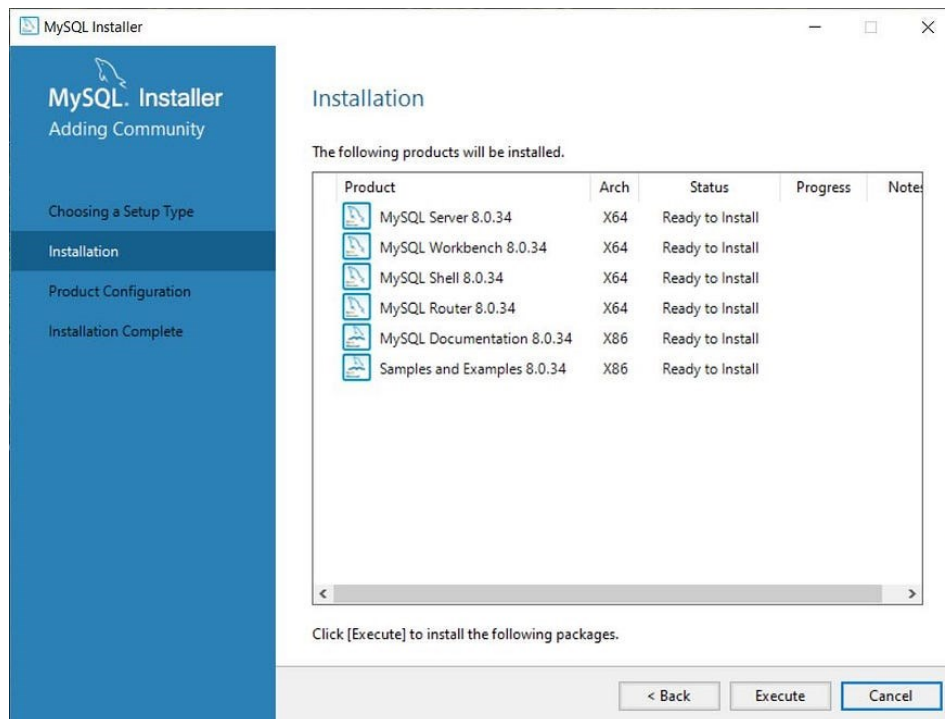
- In the next window, just click “Execute.”



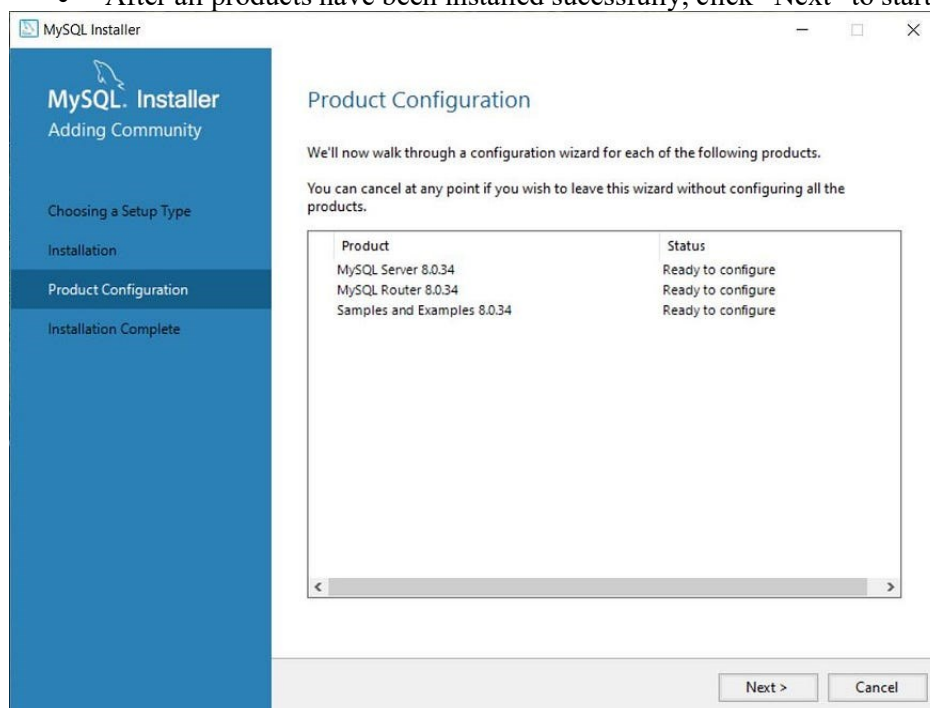
- After all products have been downloaded successfully, as indicated by the green checkmark, click “Next.”



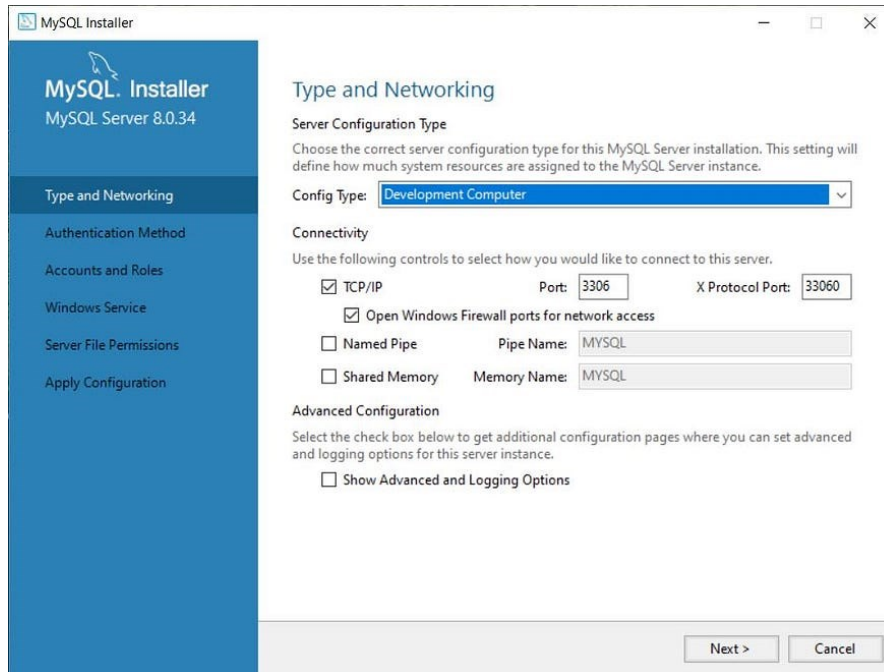
- In the next window, just click “Execute.”



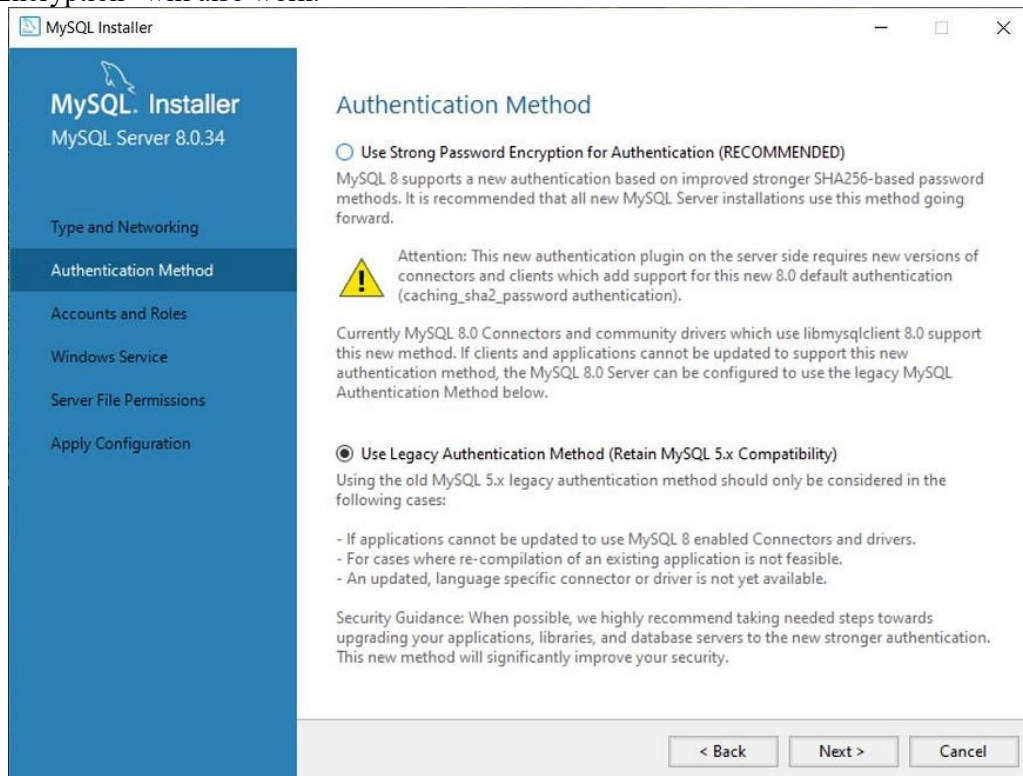
- After all products have been installed successfully, click “Next” to start the configuration.



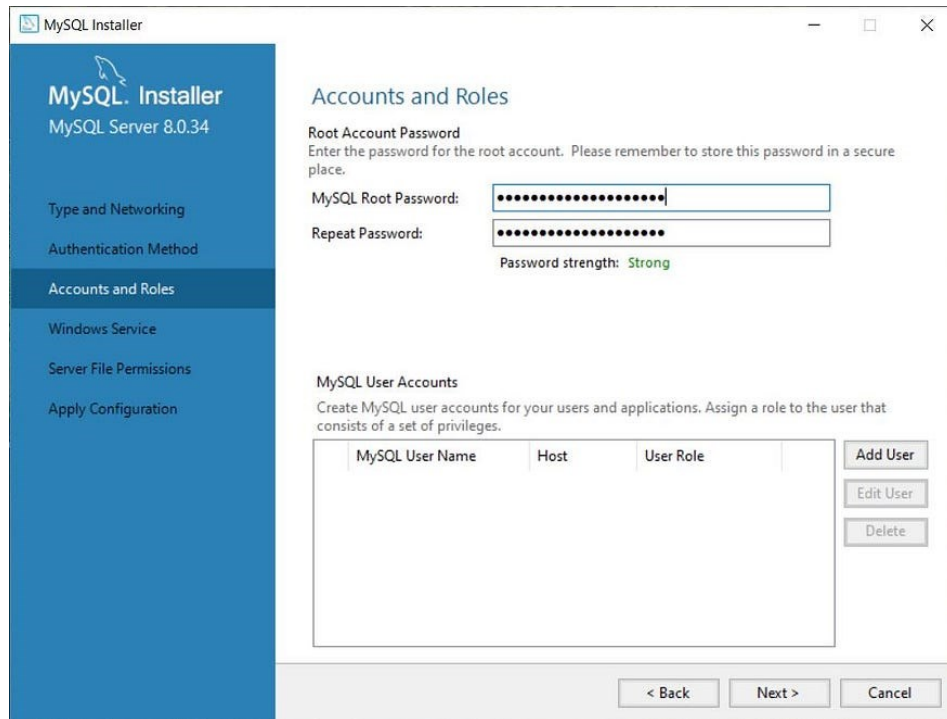
- Under “Type and Networking” choose “Development Computer” as Config Type and leave everything else as is. Then click “Next.”



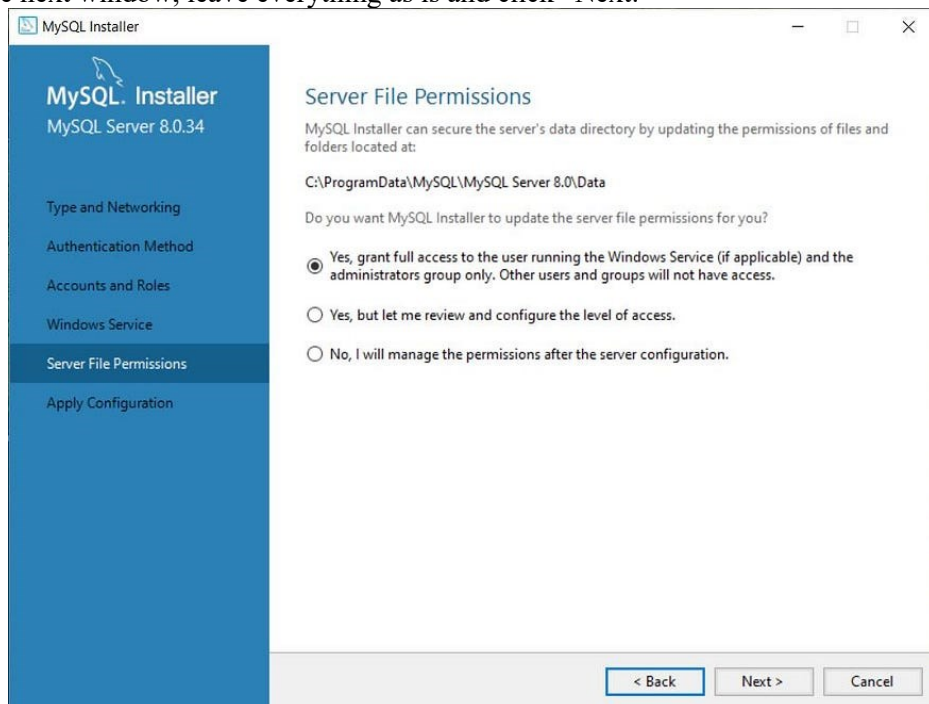
- Next, you will be asked to choose an authentication method. For increased compatibility, choose “Use Legacy Authentication Method” and click “Next.” Choosing “Use Strong Password Encryption” will also work.



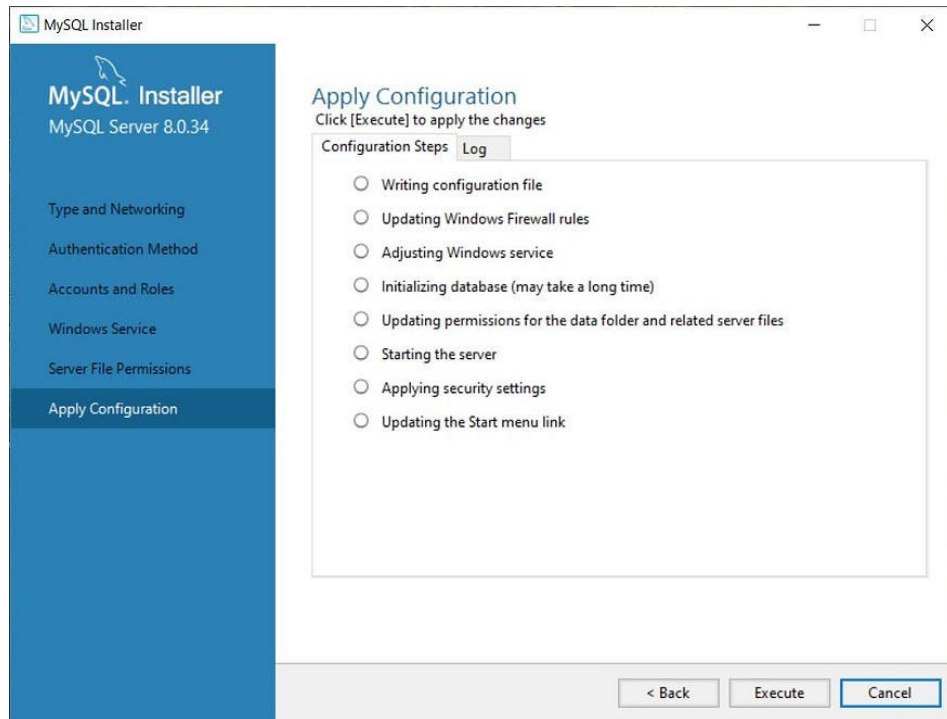
- In the “Account and Roles” window, choose a new password for your root account. Be sure to choose a strong [password](#) and enter the password under MySQL Root Password and Repeat Password then click “Next.”



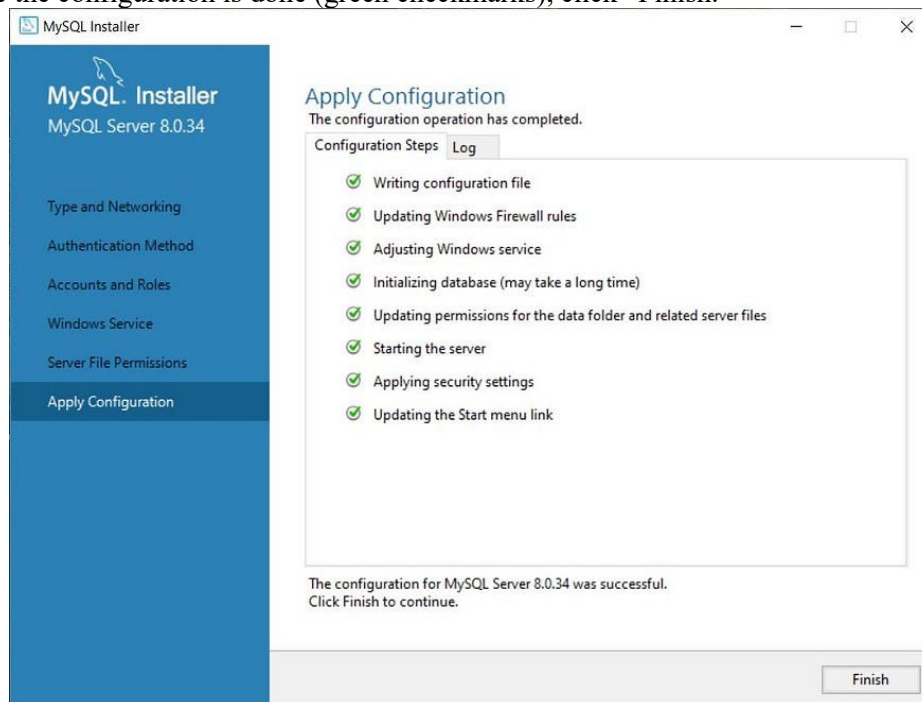
- In the next window, leave everything as is and click “Next.”



- Next, grant full access to let the Installer update server file permissions and click “Next.” Click “Execute” to apply the configuration changes.

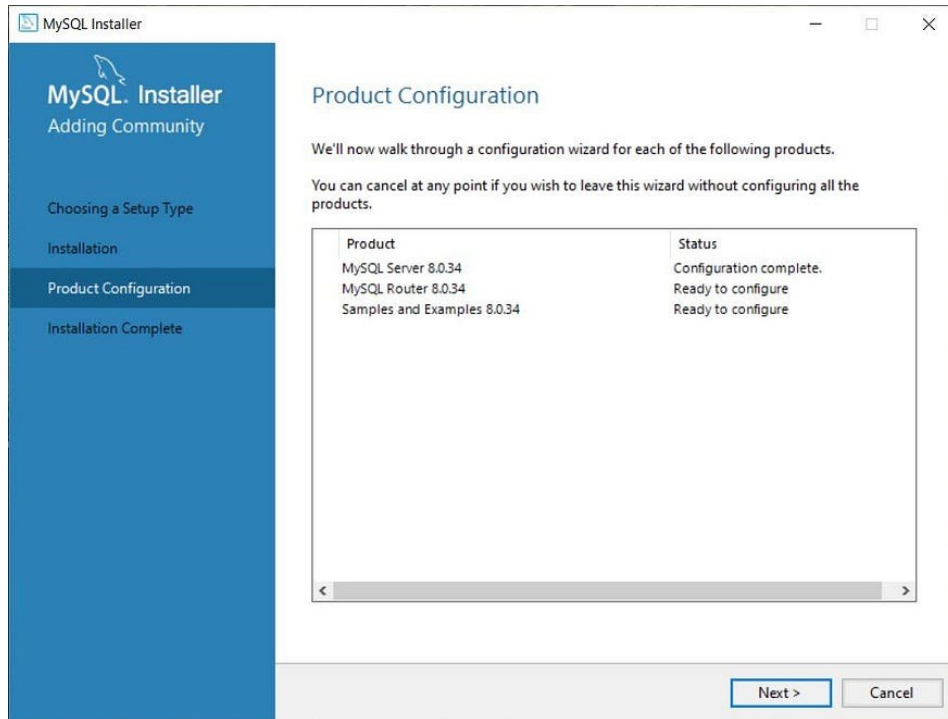


- Once the configuration is done (green checkmarks), click “Finish.”

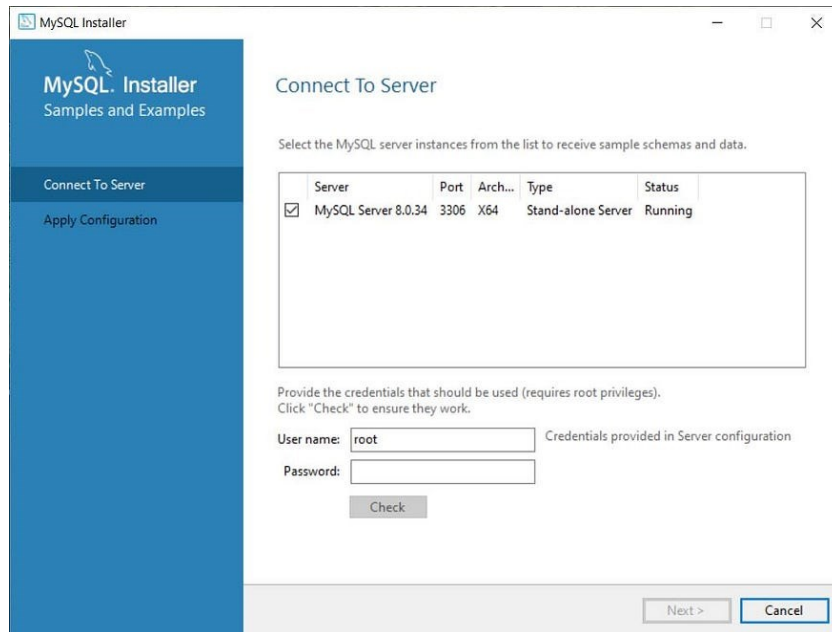


That’s the first step done. You just downloaded and installed MySQL Server and MySQL Workbench. In the next step, we will configure MySQL Router and “Samples and Examples”. Both Services are part of the full Installation. If you chose a custom installation earlier, you can skip ahead to Step 3.

- Click “Next” to start the configuration.

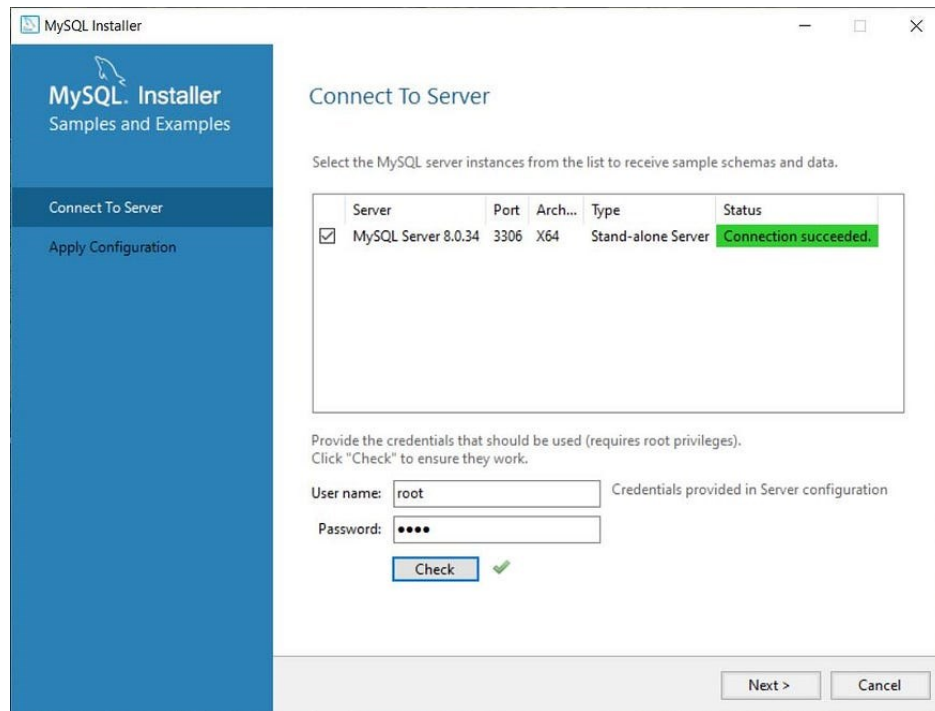


- Under “Connect To Server” enter the password you chose earlier under Password and click “Check.”

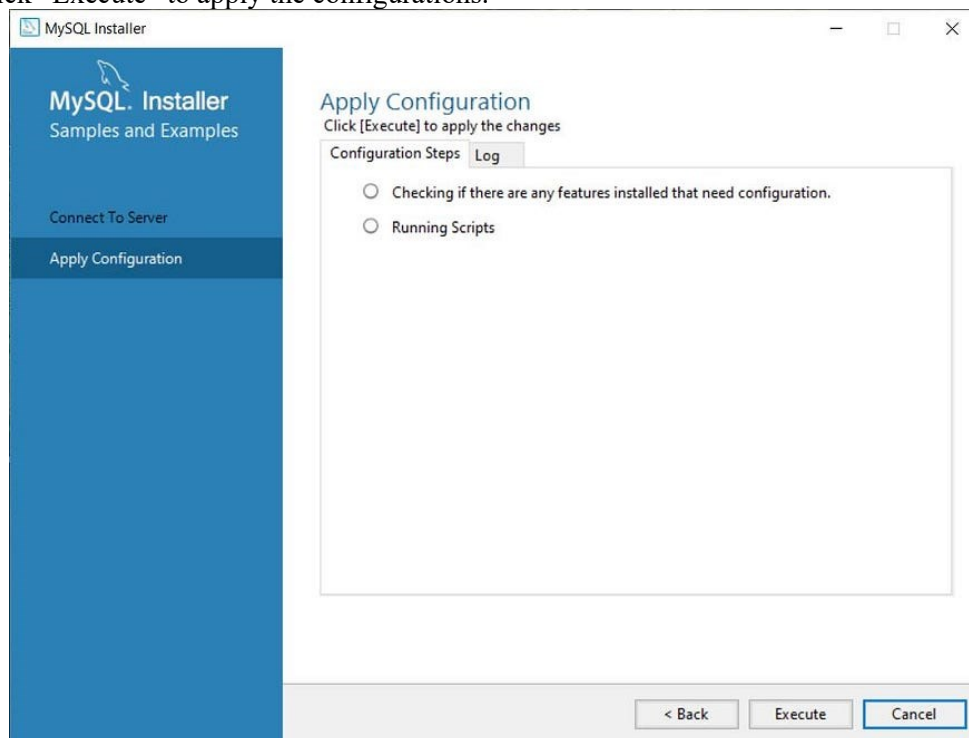


If the password was correct, a green checkmark will appear and the status of the connection will change to “Connection succeeded.”

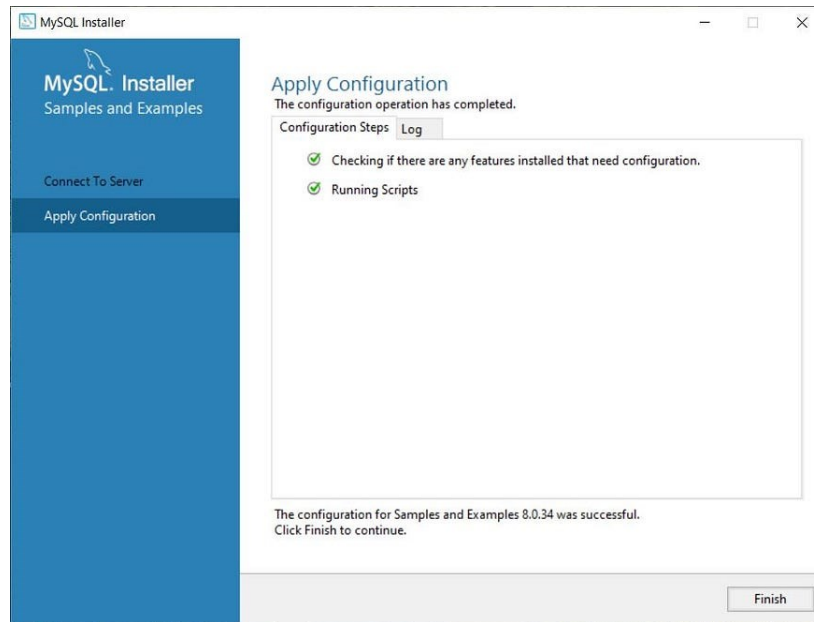
- Click “Next.”



- Click "Execute" to apply the configurations.

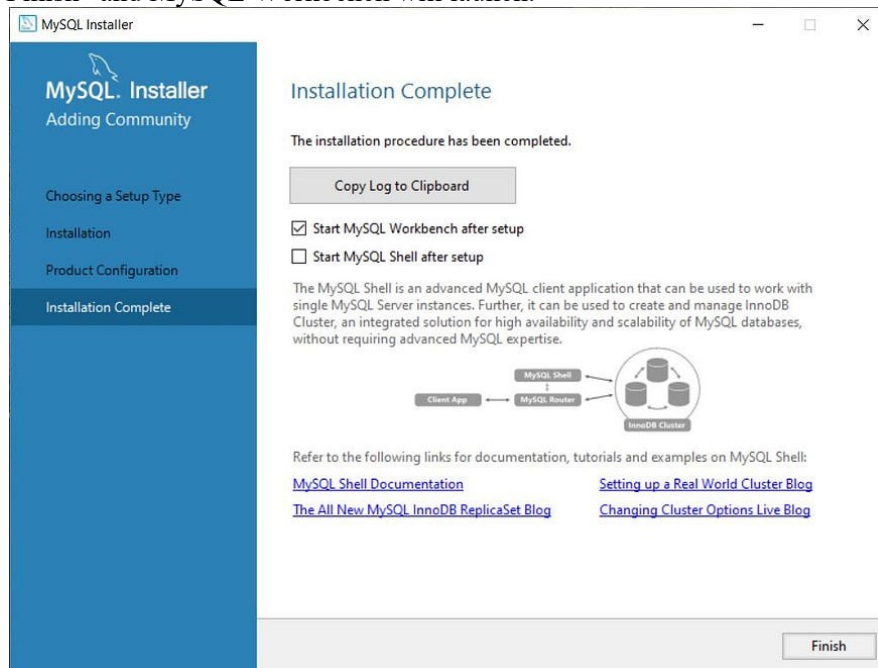


- Once the configuration is completed (green checkmarks), click "Finish."



That's it! The installation is complete. You can now choose whether MySQL Workbench and Shell should be launched — we only need MySQL Workbench right now.

- Click “Finish” and MySQL Workbench will launch.

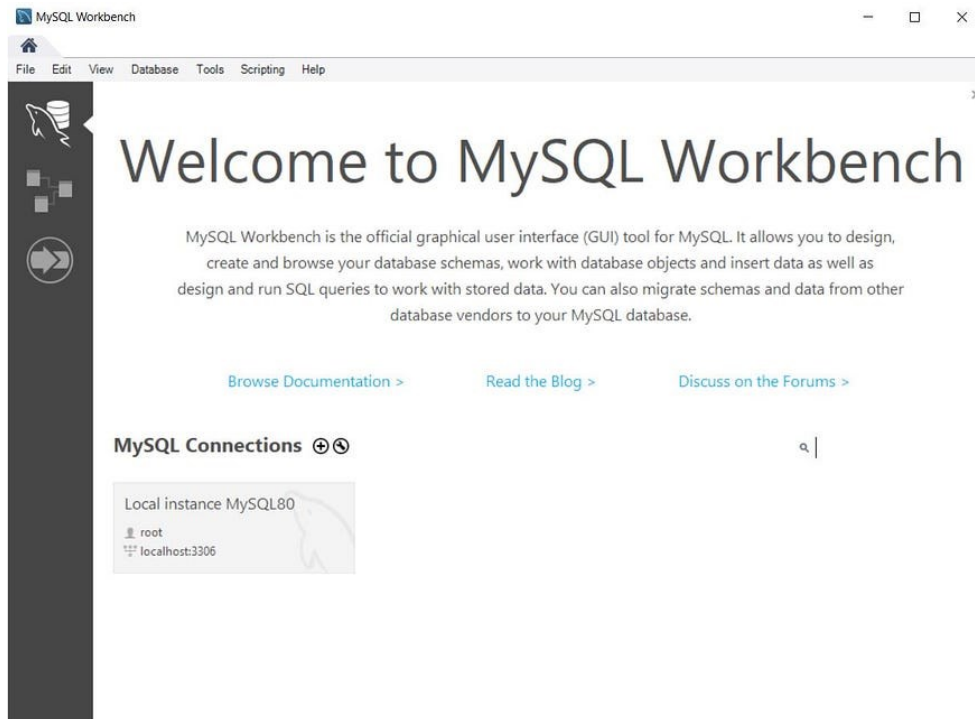


Step 3: Using MySQL Workbench

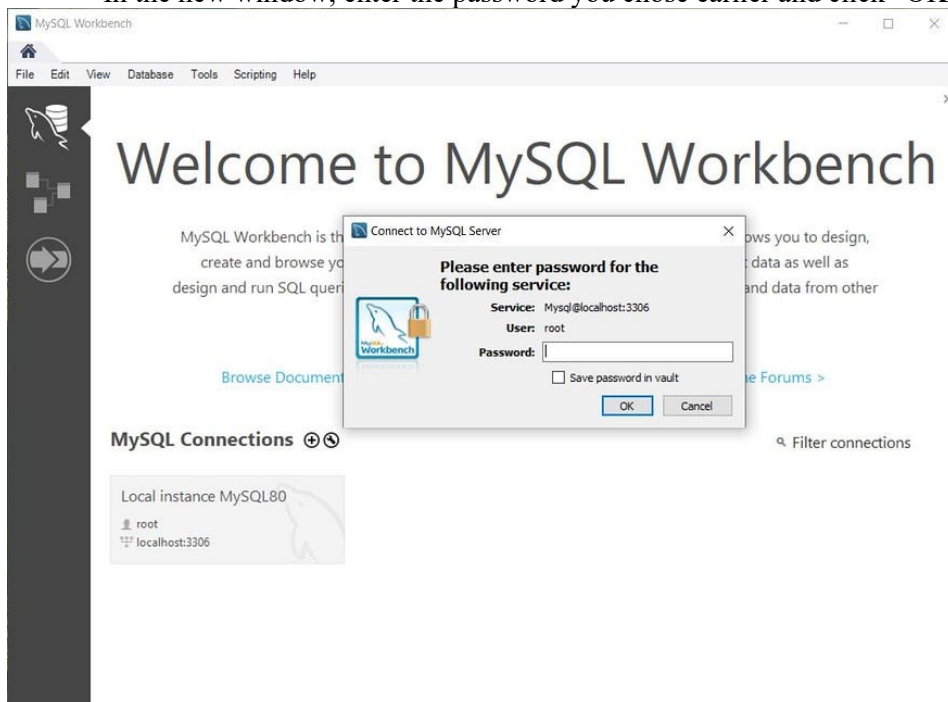
At the end of the installation process, MySQL Workbench should launch automatically. If it doesn't, start the program manually. You are greeted by the graphical user interface (GUI) tool for MySQL, the MySQL Workbench.

In the bottom half of the window under “MySQL Connections” you should see one tile for your local instance connection, Local instance MySQL80.

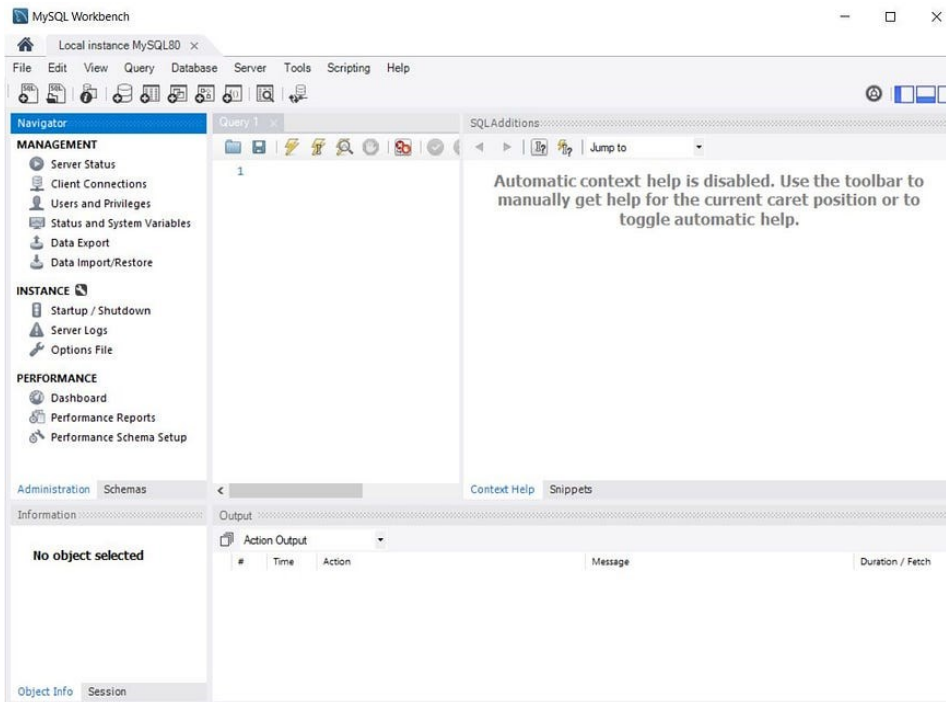
- Go ahead and double-click it.



- In the new window, enter the password you chose earlier and click “OK.”



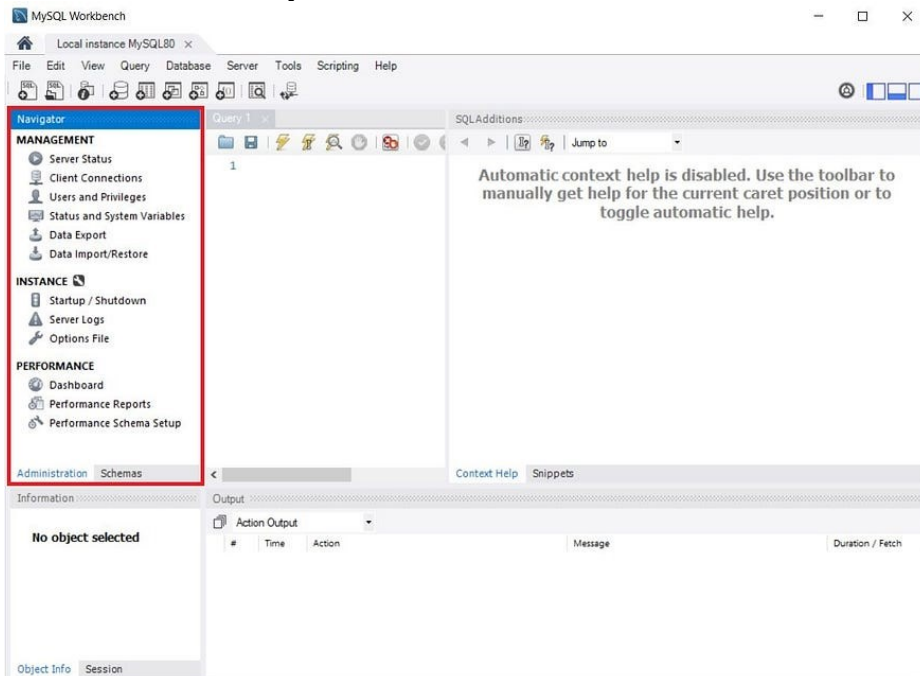
Aaaand we're in. You are now connected to your MySQL Server and can start using the MySQL Workbench.



Let's quickly look at the most important components of the user interface:

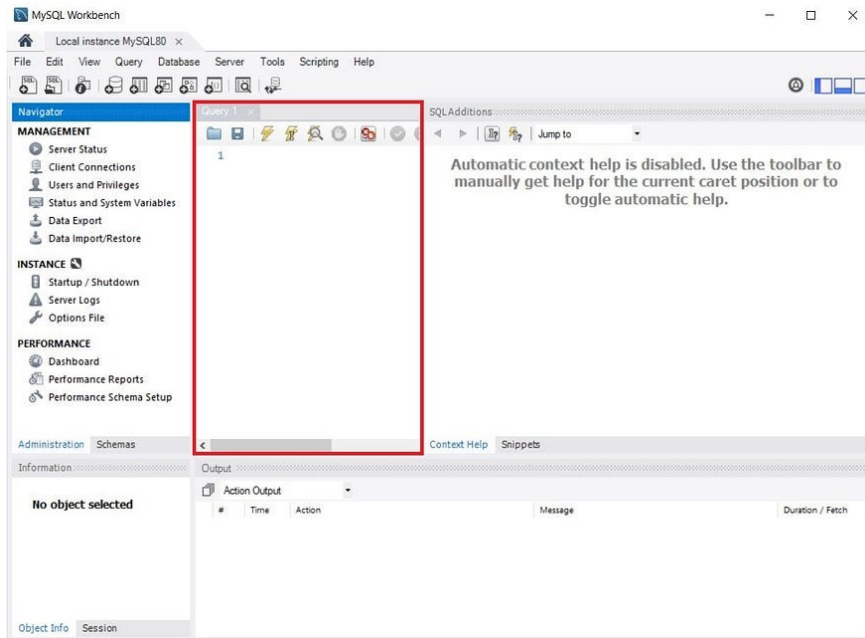
Navigator

The Navigator is typically located on the left-hand side of the MySQL Workbench GUI. It serves as a central hub for database management and navigation. It houses some administrative features and a handy tree-like structure of all your databases under the tab "Schemas."



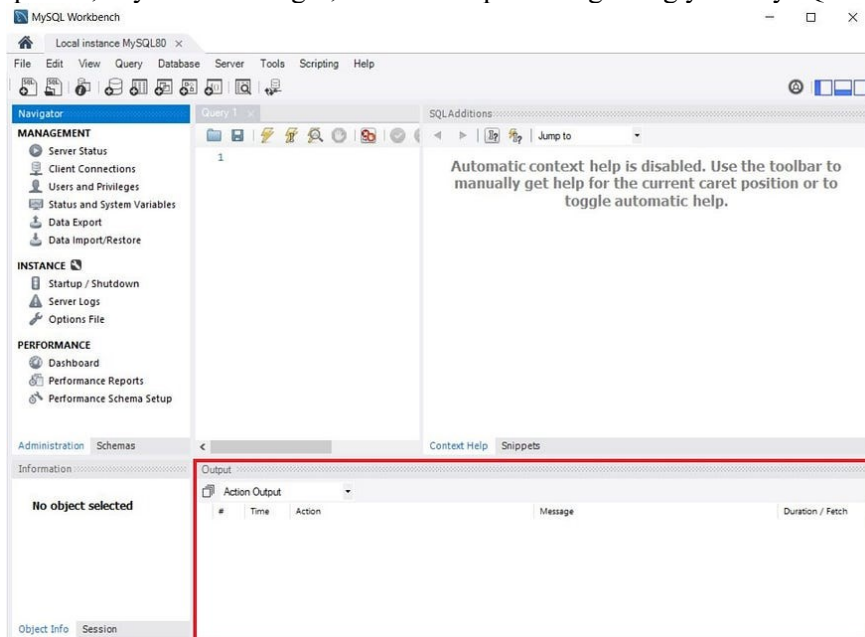
Query

The "Query" tab, located in the middle of the workspace area, is where you can write and execute SQL queries. This is where you interact with your MySQL databases using SQL commands.



Output

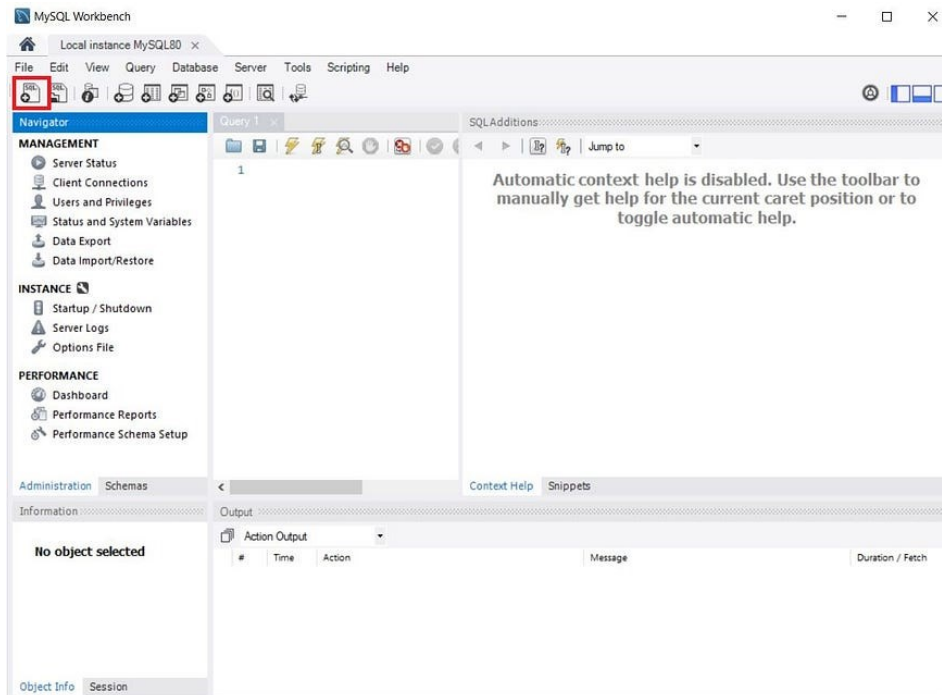
The “Output” tab, usually positioned at the bottom of the MySQL Workbench interface, displays various types of information and feedback. This includes feedback on the (successful) execution of your SQL queries, any error messages, and status updates regarding your MySQL server.



Step 4: Testing the Connection

To test your connection, use the following steps:

- In MySQL Workbench, open a new query tab by clicking the “+ SQL” icon in the top left corner or selecting “File” > “New Query Tab.”

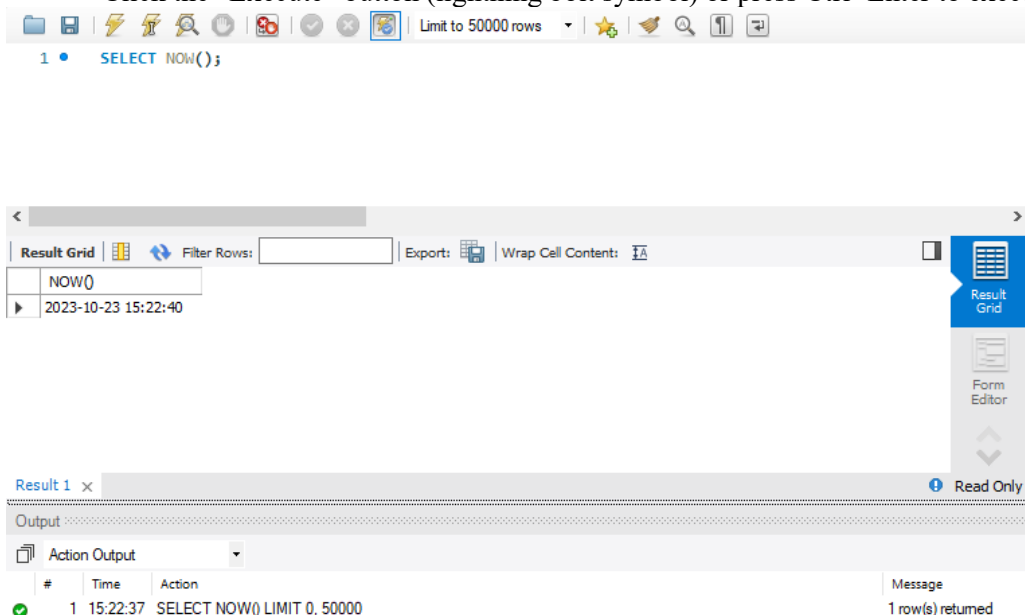


You can use a basic SQL query to test the connection. For example, you can run a query to display the current date and time:

```
SELECT NOW();
```

This query will retrieve the current date and time from the MySQL server. If you receive a result with the current date and time, it confirms that your connection to the MySQL server is working.

- Enter `SELECT NOW()` into the new query tab.
- Click the “Execute” button (lightning bolt symbol) or press `Ctrl+Enter` to execute the query.

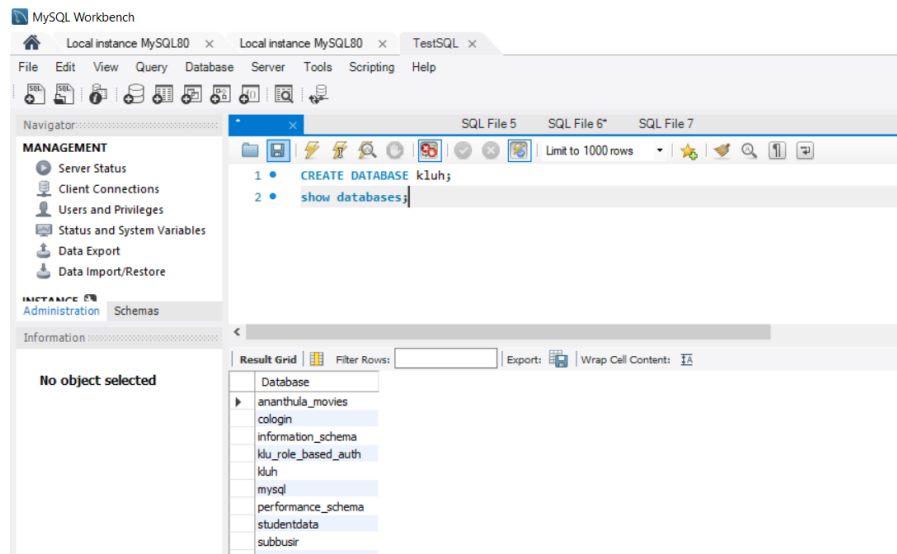


If the query executes successfully and you see the current date and time in the result pane, it means that your connection to the MySQL server is functioning correctly. If you get an error or don't see any results, it could indicate a connection issue. In that case, double-check your connection settings, including the hostname, port, username, and password, and ensure your MySQL server is running.

Database Schema & Queries (MySQL)

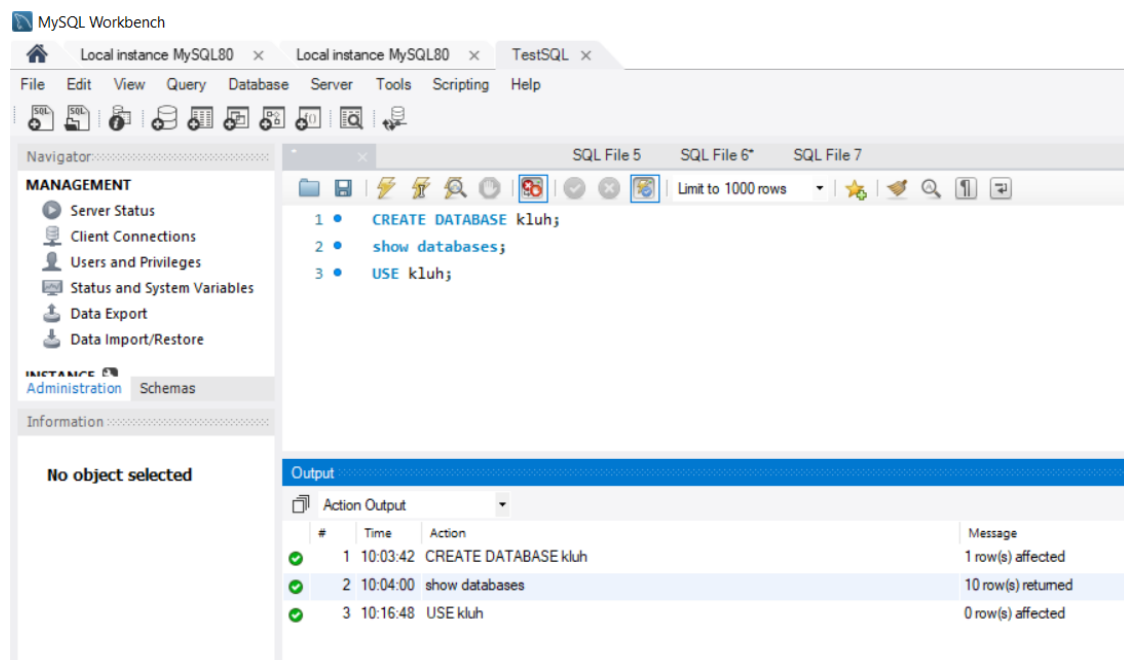
Create Database

CREATE DATABASE kluh;



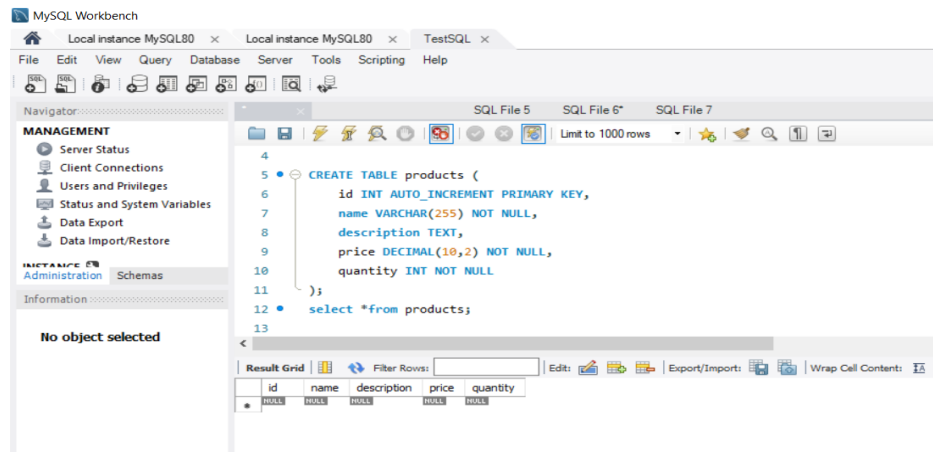
Use Database

USE kluh;



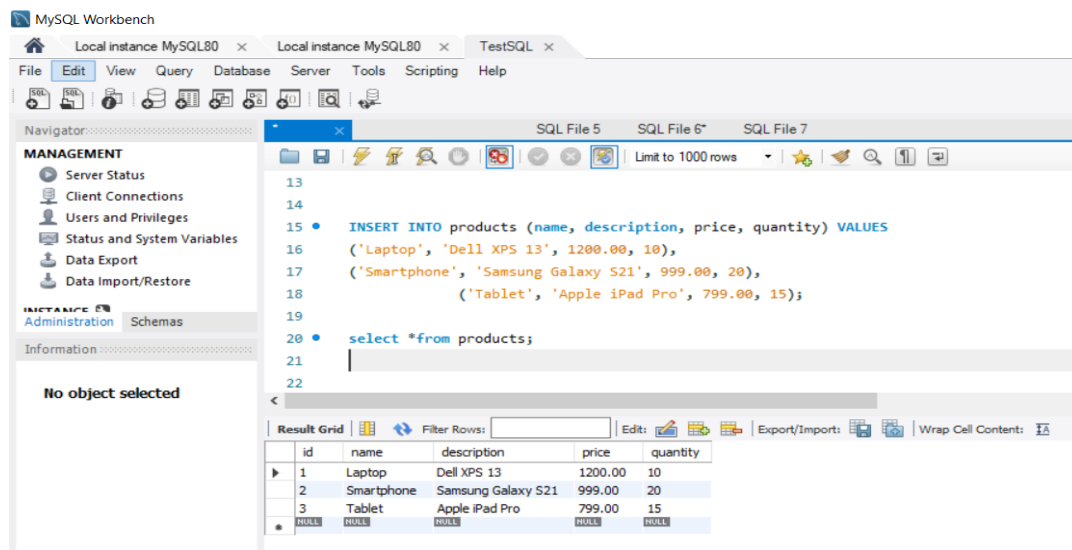
Create Table

```
CREATE TABLE products (  
  id INT AUTO_INCREMENT PRIMARY KEY,  
  name VARCHAR(255) NOT NULL,  
  description TEXT,  
  price DECIMAL(10,2) NOT NULL,  
  quantity INT NOT NULL  
);
```



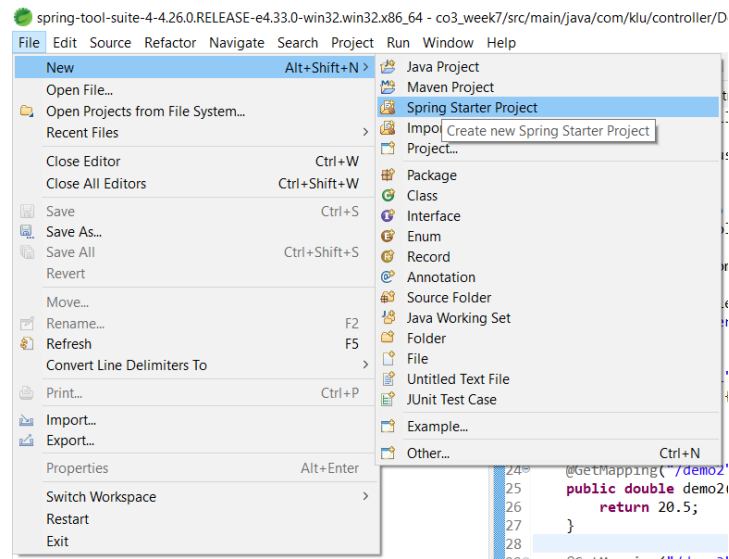
Insert Sample Data

```
INSERT INTO products (name, description, price, quantity) VALUES  
( 'Laptop', 'Dell XPS 13', 1200.00, 10),  
( 'Smartphone', 'Samsung Galaxy S21', 999.00, 20),  
( 'Tablet', 'Apple iPad Pro', 799.00, 15);
```



Project creation:

Step 1:



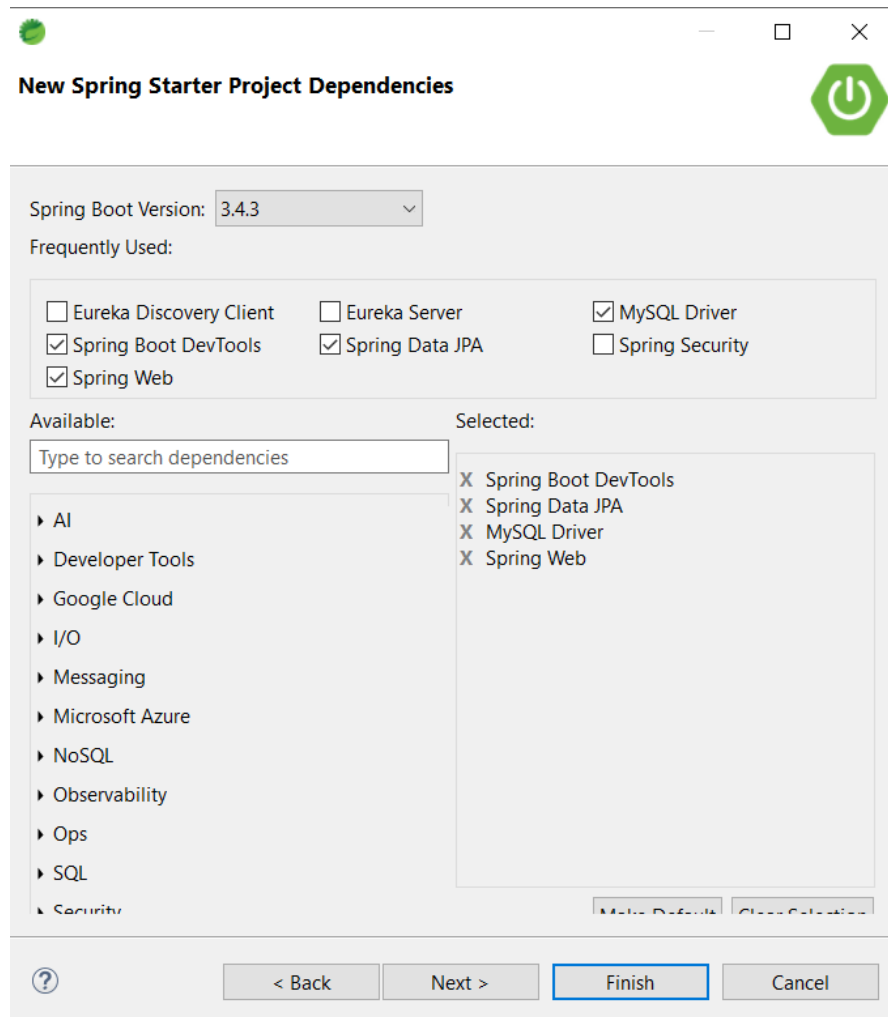
Step2

The 'New Spring Starter Project' dialog box is shown. It contains the following fields and options:

- Service URL: <https://start.spring.io>
- Name: `week8_co3`
- ☒ Use default location
- Location: `C:\files\springboot\spring-tool-suite-4-4.26.0.RELEASE-e4.33.0-win3` (with a 'Browse' button)
- Type: `Maven` (dropdown)
- Packaging: `Jar` (dropdown)
- Java Version: `17` (dropdown)
- Language: `Java` (dropdown)
- Group: `com.klu`
- Artifact: `week8_co3`
- Version: `0.0.1-SNAPSHOT`
- Description: `Demo project for Spring Boot`
- Package: `com.klu`
- Working sets section:
 - ☐ Add project to working sets (with a 'New...' button)
 - Working sets: (dropdown menu) (with a 'Select...' button)

At the bottom, there are navigation buttons: '< Back', 'Next >', 'Finish', and 'Cancel'. The 'Next >' button is highlighted.

Step 3



The dialog box is titled "New Spring Starter Project Dependencies". It features a "Spring Boot Version" dropdown set to "3.4.3". Below this is a "Frequently Used:" section with checkboxes for "Eureka Discovery Client", "Eureka Server", "MySQL Driver", "Spring Boot DevTools", "Spring Data JPA", and "Spring Security". The "MySQL Driver", "Spring Boot DevTools", "Spring Data JPA", and "Spring Web" checkboxes are checked. Below this is an "Available:" section with a search bar and a list of categories: AI, Developer Tools, Google Cloud, I/O, Messaging, Microsoft Azure, NoSQL, Observability, Ops, SQL, and Security. To the right is a "Selected:" section with a list of dependencies: "X Spring Boot DevTools", "X Spring Data JPA", "X MySQL Driver", and "X Spring Web". At the bottom are buttons for "< Back", "Next >", "Finish", and "Cancel".

Spring Boot Version: 3.4.3

Frequently Used:

☐ Eureka Discovery Client ☐ Eureka Server ☒ MySQL Driver
☒ Spring Boot DevTools ☒ Spring Data JPA ☐ Spring Security
☒ Spring Web

Available:

Type to search dependencies

- AI
- Developer Tools
- Google Cloud
- I/O
- Messaging
- Microsoft Azure
- NoSQL
- Observability
- Ops
- SQL
- Security

Selected:

- X Spring Boot DevTools
- X Spring Data JPA
- X MySQL Driver
- X Spring Web

< Back Next > Finish Cancel

```
week8_co3 [boot] [devtools]
├── src/main/java
│   ├── com.klu
│   │   ├── com.klu.controller
│   │   │   ├── ProductController.java
│   │   ├── com.klu.entity
│   │   │   ├── Product.java
│   │   ├── com.klu.repository
│   │   │   ├── ProductRepository.java
│   │   ├── com.klu.service
│   │   │   ├── ProductService.java
│   │   │   └── ProductServiceImpl.java
│   └── src/main/resources
│       ├── static
│       ├── templates
│       └── application.properties
```

Programs:

ProductController.java

```
package com.klu.controller;

import com.klu.entity.Product;
import com.klu.service.ProductService;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;

import java.util.List;

@RestController
@RequestMapping("/api/products")
public class ProductController {

    private final ProductService productService;

    // Setter-based Dependency Injection
    public ProductController(ProductService productService) {
        this.productService = productService;
    }

    @GetMapping
    public List<Product> getAllProducts() {
        return productService.getAllProducts();
    }

    @GetMapping("/{id}")
    public ResponseEntity<Product> getProductById(@PathVariable Long id) {
        return ResponseEntity.ok(productService.getProductById(id));
    }

    @PostMapping
    public ResponseEntity<Product> createProduct(@RequestBody Product product) {
        return ResponseEntity.ok(productService.saveProduct(product));
    }

    @PutMapping("/{id}")
    public ResponseEntity<Product> updateProduct(@PathVariable Long id, @RequestBody
    Product productDetails) {
        return ResponseEntity.ok(productService.updateProduct(id, productDetails));
    }

    @DeleteMapping("/{id}")
    public ResponseEntity<Void> deleteProduct(@PathVariable Long id) {
        productService.deleteProduct(id);
    }
}
```

```

        return ResponseEntity.noContent().build();
    }
}

```

Product.java:

```

package com.klu.entity;

import jakarta.persistence.*;

@Entity
@Table(name = "products")
public class Product {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String name;
    private String description;
    private double price;
    private int quantity;

    // Constructors
    public Product() {}

    public Product(String name, String description, double price, int quantity) {
        this.name = name;
        this.description = description;
        this.price = price;
        this.quantity = quantity;
    }

    // Getters and Setters
    public Long getId() { return id; }
    public void setId(Long id) { this.id = id; }

    public String getName() { return name; }
    public void setName(String name) { this.name = name; }

    public String getDescription() { return description; }
    public void setDescription(String description) { this.description = description; }

    public double getPrice() { return price; }
    public void setPrice(double price) { this.price = price; }

    public int getQuantity() { return quantity; }
    public void setQuantity(int quantity) { this.quantity = quantity; }
}

```

ProductRepository.java

```
package com.klu.repository;

import com.klu.entity.Product;
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.stereotype.Repository;

@Repository
public interface ProductRepository extends JpaRepository<Product, Long> {
}
```

ProductService.java

```
package com.klu.service;

import com.klu.entity.Product;
import java.util.List;

public interface ProductService {
    List<Product> getAllProducts();
    Product getProductById(Long id);
    Product saveProduct(Product product);
    Product updateProduct(Long id, Product product);
    void deleteProduct(Long id);
}
```

ProductServiceImpl.java

```
package com.klu.service;

import com.klu.entity.Product;
import com.klu.repository.ProductRepository;
import org.springframework.stereotype.Service;

import java.util.List;
import java.util.Optional;

@Service
public class ProductServiceImpl implements ProductService {

    private final ProductRepository productRepository;

    // Constructor-based Dependency Injection
    public ProductServiceImpl(ProductRepository productRepository) {
        this.productRepository = productRepository;
    }
}
```

```

@Override
public List<Product> getAllProducts() {
    return productRepository.findAll();
}

@Override
public Product getProductById(Long id) {
    return productRepository.findById(id)
        .orElseThrow(() -> new RuntimeException("Product not found"));
}

@Override
public Product saveProduct(Product product) {
    return productRepository.save(product);
}

@Override
public Product updateProduct(Long id, Product productDetails) {
    Product existingProduct = getProductById(id);
    existingProduct.setName(productDetails.getName());
    existingProduct.setDescription(productDetails.getDescription());
    existingProduct.setPrice(productDetails.getPrice());
    existingProduct.setQuantity(productDetails.getQuantity());
    return productRepository.save(existingProduct);
}

@Override
public void deleteProduct(Long id) {
    productRepository.deleteById(id);
}
}

```

Application.properties:

```

spring.application.name=week8_co3
spring.datasource.url=jdbc:mysql://localhost:3306/kluh
spring.datasource.username=root
spring.datasource.password=root
spring.jpa.database-platform=org.hibernate.dialect.MySQLDialect
spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=true
server.port=8989

```

API Endpoints

HTTP Method	Endpoint	Description
GET	http://localhost:8989/api/products	Get all products
GET	http://localhost:8989/api/products/{id}	Get product by ID
POST	http://localhost:8989/api/products	Create new product

HTTP Method	Endpoint	Description
PUT	http://localhost:8989/api/products/{id}	Update existing product
DELETE	http://localhost:8989/api/products/{id}	Delete a product

Outputs:

```

localhost:8989/api/products

Pretty-print ☐

[{"id":1,"name":"Laptop","description":"Dell XPS 13","price":1200.0,"quantity":10},
{"id":2,"name":"Smartphone","description":"Samsung Galaxy S21","price":999.0,"quantity":20},
{"id":3,"name":"Tablet","description":"Apple iPad Pro","price":799.0,"quantity":15}]

```

Postman testing:

1. Using get method

The screenshot shows the Postman interface with a GET request to `http://localhost:8989/api/products`. The response is a 200 OK status with a response time of 86 ms and a body size of 423 B. The response body is displayed in JSON format, showing an array of three product objects:

```

{
  "id": 1,
  "name": "Laptop",
  "description": "Dell XPS 13",
  "price": 1200.0,
  "quantity": 10
},
{
  "id": 2,
  "name": "Smartphone",
  "description": "Samsung Galaxy S21",
  "price": 999.0,
  "quantity": 20
},
{
  "id": 3,
  "name": "Tablet",
  "description": "Apple iPad Pro",
  "price": 799.0,
  "quantity": 15
}

```

Using post method:

The screenshot shows a REST client interface with a POST request to `http://localhost:8989/api/products`. The request body is a JSON object:

```
{
  "id": 1,
  "name": "wifirouter",
  "description": "dlink",
  "price": 11200.0,
  "quantity": 98
}
```

The response status is **200 OK** with a response time of 648 ms and a body size of 244 B. The response body is also shown as a JSON object:

```
{
  "id": 1,
  "name": "wifirouter",
  "description": "dlink",
  "price": 11200.0,
  "quantity": 98
}
```

After post method ,Database table updated :

The screenshot shows a database client interface with a SQL query `select *from products;` executed. The result is displayed in a table with 5 columns: `id`, `name`, `description`, `price`, and `quantity`.

	id	name	description	price	quantity
1	1	wifirouter	dlink	11200	98
2	2	Smartphone	Samsung Galaxy S21	999	20
3	3	Tablet	Apple iPad Pro	799	15
*	NULL	NULL	NULL	NULL	NULL

Viva:

1. Can you explain the role of each layer (Controller, Service, Repository) in a Spring Boot MVC application, especially in the context of CRUD operations?
2. How would you configure and connect a Spring Boot application to a relational database, and what dependencies are necessary for CRUD operations?
3. Describe how you would create and map a JPA entity for a table in the database. How does this mapping support CRUD operations?
4. How do you handle data validation in a Spring Boot CRUD application before saving data to the database? Can you give examples of annotations used for validation?

5. What is the purpose of `@Transactional` in Spring Boot, and how does it ensure data consistency during CRUD operations?

Solutions:

1. Can you explain the role of each layer (Controller, Service, Repository) in a Spring Boot MVC application, especially in the context of CRUD operations?

The **Controller layer** handles HTTP requests and responses, mapping them to service methods. The **Service layer** implements business logic and interacts with the repository. The **Repository layer** communicates with the database using Spring Data JPA, allowing CRUD operations like `save()`, `findById()`, and `deleteById()`.

2. How would you configure and connect a Spring Boot application to a relational database, and what dependencies are necessary for CRUD operations?

Add dependencies like **Spring Boot Starter Data JPA** and **MySQL/PostgreSQL Driver** in `pom.xml`. Configure `application.properties` with database credentials and set `spring.jpa.hibernate.ddl-auto=update` for automatic schema management.

3. Describe how you would create and map a JPA entity for a table in the database. How does this mapping support CRUD operations?

Use `@Entity` and `@Table` to define a JPA entity representing a database table. Each column is mapped using `@Column`, and `@Id` with `@GeneratedValue` ensures unique identifiers. The **JpaRepository** interface provides built-in CRUD operations.

4. How do you handle data validation in a Spring Boot CRUD application before saving data to the database? Can you give examples of annotations used for validation?

Spring Boot uses **Bean Validation API** (`javax.validation`). Annotations like `@NotBlank`, `@Size`, `@Min`, and `@Max` enforce constraints. `@Valid` in the controller ensures validation before processing the request.

5. What is the purpose of `@Transactional` in Spring Boot, and how does it ensure data consistency during CRUD operations?

`@Transactional` ensures that multiple database operations execute as a single unit. If one operation fails, all changes are rolled back, maintaining data consistency.

Aim: Spring Boot with Rest API and CRUD Operations

